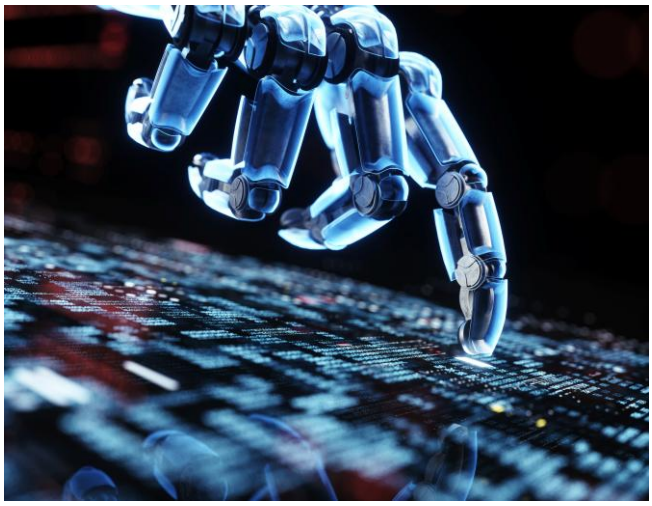


Natural Language Processing Project: Automated Essay Scoring

COMP316

Tahsheel Briglal – 223149731

Jeryn Jadav Naidoo – 223009245

Tashmika Pillay – 223097300

Introduction

Automated Essay Scoring is an application of Natural Language Processing (NLP) that evaluates and scores written essays based on a model. Traditionally, essay grading requires significant human effort, time, and subjectivity, resulting in Automated Essay Scoring emerging as a promising solution to address these challenges. This project explores the use of machine learning to predict essay scores based on linguistic features extracted from the text. The ability to automate essay scoring has significant implications for education technology, standardized testing and personalized learning.

Project Objective

The primary objective is to develop and compare different machine learning models for essay scoring and evaluate their performance using metrics such as F1 score and Cohen's Kappa. Firstly, we preprocessed and vectorized essay text in order to fit the models. We then used the preprocessed and vectorized essay text to train each model. The models used for comparison include Logistic Regression, Random Forest, and Naïve Bayes. All the models will be evaluated based on their performance and the optimal model will be selected based on its ability to generalize and accurately predict essay scores.

Natural language processing techniques & resources used

1. Tokenization

This involves splitting the essay text into individual tokens, which are typically words. We used NLTK's *word_tokenize* function to convert each essay into a list of lowercase word tokens.

2. Removal of Stopwords

We made use of NLTK's *stopword* corpus to remove common English stopwords, for example, "and", “is” and "this", which do not contribute much semantic value. This refined the data, enhancing the quality of features.

3. Lemmatization

NLTK's *WordNetLemmatizer* reduces each word to its root word to ensure the different grammatical forms of a word are treated equally.

4. TF – IDF Vectorization

Term Frequency-Inverse Document Frequency is a method that weighs words based on their frequency in a specific essay relative to their appearance across the entire corpus. Additionally,

unigrams and bigrams were used to capture the word combinations that may influence essay scoring.

5. Label Encoding

The label encoding involves locating all categorical scores (e.g. 'A', 'B', 'C') and converts them into numerical scores (e.g. 'A' becomes 0, 'B' becomes 1, and so forth) for these scores to be used in the model building and training. To do this we used *LabelEncoder* from *sklearn.preprocessing*, which forms the basis of transforming the unstructured essay text into a structured input suitable for machine learning algorithms.

Resources

- Programming Language: Python
- Libraries & API's:
    - *nltk* – for text preprocessing
    - *scikit-learn* – for machine learning, TF-IDF & evaluation
    - *pandas & numpy* – for data handling
    - *matplotlib & seaborn* – for visualization
- Dataset: A labeled dataset of essays from Learning Agency Lab

Setting up environment

```
[ ]  # Install required packages
     !pip install nltk pandas numpy scikit-learn seaborn

     # Import necessary libraries
     import nltk
     import pandas as pd
     import numpy as np
     import matplotlib.pyplot as plt
     import seaborn as sns
     from sklearn.model_selection import train_test_split
     from sklearn.feature_extraction.text import TfidfVectorizer
     from sklearn.linear_model import LogisticRegression
     from sklearn.metrics import f1_score, cohen_kappa_score, classification_report
     from sklearn.preprocessing import LabelEncoder
     from nltk.corpus import stopwords
     from nltk.tokenize import word_tokenize
     from nltk.stem import WordNetLemmatizer

     # Download NLTK resources
     nltk.download('punkt_tab')
     nltk.download('stopwords')
     nltk.download('wordnet')
     nltk.download('omw-1.4')
```

The screenshot above shows how we installed the necessary libraries using pip to implement the NLP techniques mentioned previously.

Methodology

Text cleaning & preprocessing

Loading and Understanding dataset

```
# Load the dataset
from google.colab import files
uploaded = files.upload()

# Read the dataset
df = pd.read_csv('train.csv')

# Explore the dataset
print(df.head())

print(df['score'].value_counts())
```

```
Choose Files  No file chosen    Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving train.csv to train (1).csv
  essay_id                                          full_text  score
0  000d118  Many people have car where they live. The thin...      3
1  000fe60  I am a scientist at NASA that is discussing th...      3
2  001ab80  People always wish they had the same technolog...      4
3  001bdc0  We all heard about Venus, the planet without a...      4
4  002ba53  Dear, State Senator\n\nThis is a letter to arg...      3
score
3    6280
2    4723
4    3926
1    1252
5     970
6     156
Name: count, dtype: int64
```

Dataset Overview

We used *files.upload ( )* to upload the CSV dataset. *pd.read_csv( )* loaded the uploaded dataset into a *Pandas DataFrame(df)*, to structure a tabular format for data analysis. The use of *df.head( )* provided a preview of the dataset which allowed us to decide on how to model the data.

- essay_id: a unique identifier for each essay
- full_text: raw text of the essay
- score: the assigned score ranging from 1 to 6

df ['score'].value_counts( ) computed the frequency distribution of the score column, indicating which essays belonged to each score category.

- A score of 3 was the most frequent for 6280 essays
- A score of 6 was the least frequent for 156 essays

This implied the distribution was imbalanced, which could bias the model toward predicting frequent scores if not addressed. This led to requiring class weighting during model training.

Data Preprocessing

```
# Initialize NLTK tools
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess_text(text):
    # Tokenization
    tokens = word_tokenize(text.lower())

    # Remove stopwords and non-alphabetic tokens
    tokens = [lemmatizer.lemmatize(token) for token in tokens
              if token.isalpha() and token not in stop_words]

    return ' '.join(tokens)

# Apply preprocessing to essays
df['processed_text'] = df['full_text'].apply(preprocess_text)

# Encode the scores (if they're categorical)
le = LabelEncoder()
df['score_encoded'] = le.fit_transform(df['score'])
```

NLTK initialization

The code above is used for preprocessing the essay dataset in preparation for training our model. We started the process by initializing essential NLTK components, consisting of a list of *stopwords* and a *WordNet* lemmatizer, which is used to reduce words to their base forms. A preprocessing function was then used to clean essay texts by converting each essay to lowercase, tokenizing the text into individual words, removing *stopwords* and non-alphabetic tokens, and lemmatizing the remaining words. The cleaned tokens were joined to a single string, and this function was applied to each entry in the *full_text* column of the dataset. The results were stored in a new column named *processed_text* which allowed us to prepare the target for modelling. The code employed a *LabelEncoder* to encode the score values into a numerical format, which was saved in a new column called *score_encoded*. This preprocessing ensured that both the input text and the output labels are suitable for our machine learning model.

Feature Extraction

```
[ ]  # Initialize TF-IDF Vectorizer
     tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1, 2))

     # Fit and transform the text data
     X = tfidf.fit_transform(df['processed_text'])
     y = df['score_encoded']

     # Split data into train and test sets
     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Data Splitting

The cleaned text was vectorized using *TfidfVectorizer( )*, which applied TF-IDF weighting to quantify word importance. The vectorizer was initialized with *max_features = 5000* and *ngram_range = (1,2)*, which was used to extract the most important features according to term and inverse document frequency, considering unigrams, as well as bigrams. This, essentially, captured single words and

word pairs to help identify more contextual information from the essays. The function, *fit_transform* (*df ['processed_text']*), learned the vocabulary from the preprocessed essay text and converted each essay into a numerical vector based on the TF-IDF scores. These vectors were stored in X, representing the encoded essay scores, and the target labels were stored in y as *df['score_ encoded']*. The resulting matrix, where each row represents an essay and each column, either a word or bigram with TF-IDF scores, was split into training (80%) and testing (20%) datasets. To do this, we used the *train_test_split( ),* with *random_state = 42* to ensure the split was reproducible. Essentially, this procedure transformed textual essays into their respective structured numerical representations for automated scoring.

Models

Model implementation

Three machine learning models were trained and evaluated:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import MultinomialNB

models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, multi_class='multinomial'),
    'Random Forest': RandomForestClassifier(),
    'Naive Bayes': MultinomialNB()
}
```

The three distinct machine learning models, namely Logistic Regression, Random Forest and Multinomial Naïve Bayes were assessed to measure their effectiveness in predicting the encoded essay scores based on the TF-IDF vectorized essays.

Logistic Regression

This is a linear classification that estimates class probabilities by modeling a logistic function to input features and has proved to be a well-suited representation of sparse data like TF-IDF vectors. We used this multinomial model to suit our multi-class scoring from 1 to 6. We increased *max_iter* for convergence with TF-IDF features.

Random Forest

Random forest is a model that consists of a learning method that constructs multiple decision trees and outputs the mode of their predictions. Therefore, this model offers consistency in overfitting and improving generalization. We used Random Forest as it allows us to handle imbalanced score distributions and we kept the default ensemble of 100 decision trees.

Naïve Bayes

Multinomial Naïve Bayes is a probabilistic classifier. It assumes each word feature is conditionally independent of each other given the class, and is efficient for text classification problems involving word counts. This classifier was chosen for our model as it is highly efficient for discrete word counts using TF-IDF values.

Each of these models are trained using *model.fit( )* on the training dataset *(X_train, y_train)*. We then evaluated this on the test set*(X_test)*. Predictions were generated using *model.predict( )*, before using the evaluation metrics.

```
results = []

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    f1 = f1_score(y_test, y_pred, average='macro')
    kappa = cohen_kappa_score(y_test, y_pred)

    results.append({'Model': name, 'F1 Score': f1, 'Cohen\'s Kappa': kappa})
```

Metrics

We used two metrics to evaluate the effectiveness of the model.

F1 Score

The F1 score calculates the average of precision and recall independently for each class and then takes the mean of the results, which makes it compatible with imbalanced class distributions. It did this by assigning equal weight to each class, reflecting a balanced performance across all score categories (scores 1-6). It penalized poor performance in minority classes.

Cohen's Kappa

Cohen’s Kappa measures the agreement while adjusting for chance, allowing for a more robust evaluation of multi-class settings. This metric helped us to measure the agreement between the predicted and true scores. Cohen’s Kappa adjusts for random chance so values > 0.6 indicate considerable agreement.

The results, including the model's name and their respective F1 and Kappa's scores, are appended as dictionaries to the list for accumulation of results and comparison of evaluation scores.

Model Comparison

Confusion matrices & heat maps

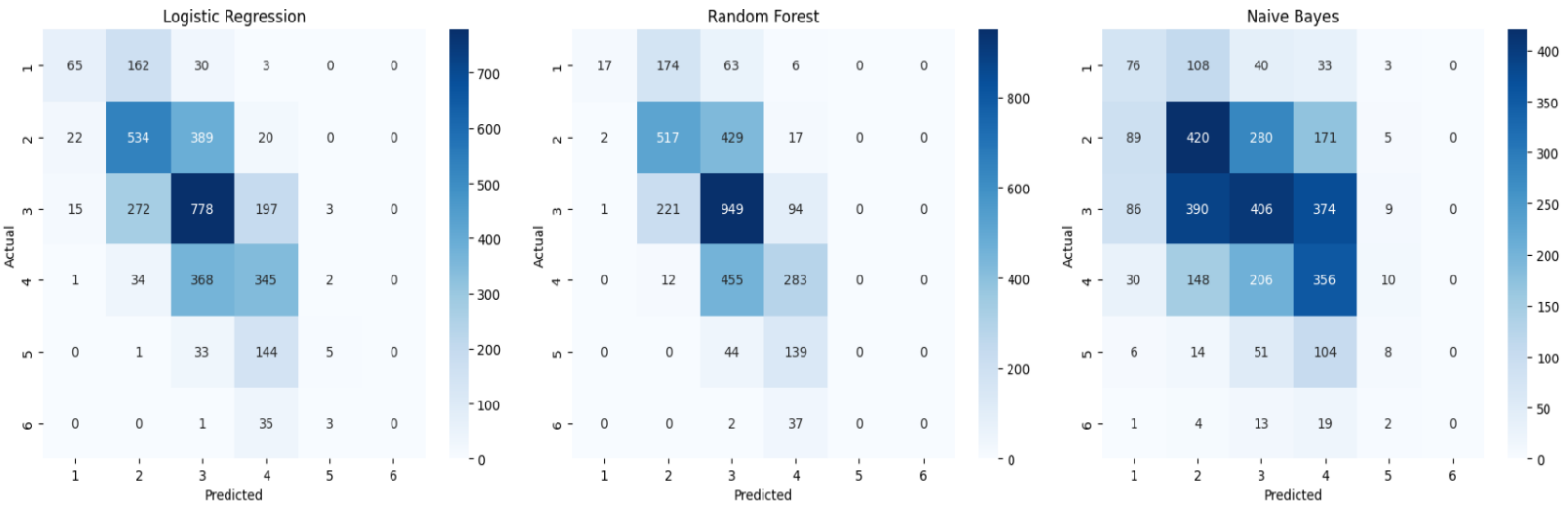
```
from sklearn.metrics import confusion_matrix

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()

for idx, (name, model) in enumerate(models.items()):
    y_pred = model.predict(X_test)
    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                ax=axes[idx], xticklabels=le.classes_, yticklabels=le.classes_)
    axes[idx].set_title(f'{name}')
    axes[idx].set_xlabel('Predicted')
    axes[idx].set_ylabel('Actual')

plt.tight_layout()
plt.show()
```

This code computes the predictions using the testing set, and displays these predicted scores against the actual scores as individual heat maps for each model.



The colour intensity suggested that higher frequencies appear a darker shade of blue. It helped us identify common error patterns, for example models frequently confusing scores within close proximity of one another. It showed the underprediction of the least frequent scores, for example a score of 6 is never predicted. For Logistic Regression, the number of times the model predicted 3 when the actual score was 3 occurred 778 times. Similarly, for Random Forest, with the same predicted and actual scores, occurred 949 times. In the Naïve Bayes heatmap, a predicted score of 2 and an actual score of 2 occurred 420 times.

Metric comparisons across each model

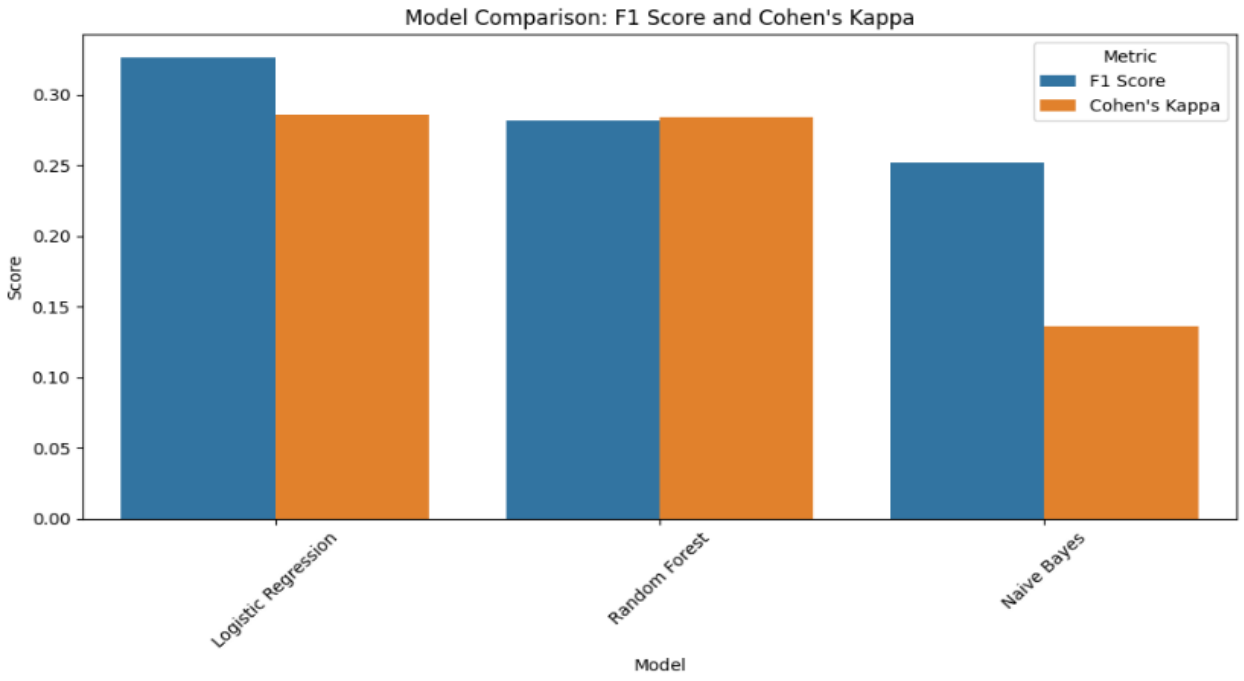
```
# Round values for better display
display_df = results_df.copy()
display_df['F1 Score'] = display_df['F1 Score'].round(3)
display_df["Cohen's Kappa"] = display_df["Cohen's Kappa"].round(3)

print(display_df.to_string(index=False))
```

```
[ ]
    results_df = pd.DataFrame(results).sort_values(by='F1 Score', ascending=False)

    plt.figure(figsize=(10, 6))
    sns.barplot(data=results_df.melt(id_vars='Model', var_name='Metric', value_name='Score'),
                x='Model', y='Score', hue='Metric')
    plt.xticks(rotation=45)
    plt.title('Model Comparison: F1 Score and Cohen\'s Kappa')
    plt.tight_layout()
    plt.show()
```

The code above created a bar plot, which compared the models according to F1 Score and Cohen's Kappa Score by first sorting the models in descending order of F1 score. It then reshaped the *DataFrame* for plotting and used Seaborn to represent the scores. We then displayed the values for precise comparison.



Model Performance Comparison

	Model	F1 Score	Cohen's Kappa
0	Logistic Regression	0.326	0.286
1	Random Forest	0.283	0.289
2	Naive Bayes	0.252	0.137

Evidently, the Logistic Regression showed a slight superiority with an F1 score of 0.326 and Kappa score of 0.286, proving it handled the imbalanced score distribution (specifically for rare scores 1 & 6) better than the other models. The model's ability to capture linear relationships between TF-IDF features and scores gave it an advantage, however the balanced class weighting only provided limited improvement.

The Random Forest model presented comparable performance with F1 score 0.283 and Kappa score 0.289, nearly matching the Logistic Regression model. This indicated it may have been more robust to misclassify extreme scores, however its tendency to overfit rare score patterns and lack of interpretability undermines this model.

On the other hand, the Naïve Bayes model underperformed with F1 score 0.252 and Kappa score 0.137, with its strong independence assumptions proving its unsuitable. The model failed to capture word co-occurrence patterns, resulting in the poorest performance across both metrics.

Evaluation

This evaluation used 3 machine learning models to predict essay scores using TF-IDF vectorized features. The models chosen for this were Logistic Regression, Random Forest, and Naive Bayes. Each model was then assessed using F1 score, to measure how well each model balanced correct predictions across the different score levels, and Cohen's Kappa, to measure how much the model's predictions agreed with the actual scores, beyond what would happen by chance.

Logistic Regression

F1 Score: 0.326 Cohen's Kappa: 0.286

Logistic Regression achieved the highest F1 score and a strong Cohen's Kappa, indicating a reliable performance in classifying essays across multiple score levels. The model effectively balanced precision and recalls across classes, handling the TF-IDF features relatively well. Its linear structure and ability to handle multiclass classification using *multi_class='multinomial'* make it a dependable option for text classification tasks. This model proved to be the best overall with regards to balance of performance and its efficiency with sparsening features.

Random Forest

F1 Score: 0.283 Cohen’s Kappa: 0.289

Random Forest slightly outperformed Logistic Regression in terms of Cohen’s Kappa, which suggested that it had a better alignment with true score distributions. Its lower F1 Score showed that it was less consistent across all classes and may have even misclassified essays in low frequency score categories. Random Forest excels in modeling complex relationships but is less suited to sparse data like TF-IDF, unless feature importance is optimized for this model.

Naïve Bayes

F1 Score: 0.252 Cohen's Kappa: 0.137

Naïve Bayes performed the poorest of both metrics. Despite this model being fast, simple to implement, and effective as a baseline, its assumption of feature independence is unrealistic in real-word text, specifically in essays where context and word order are significant. The low Cohen's Kappa score demonstrates a weak agreement with the human-assigned scores, and its trend to overpredict common classes likely contributed to its reduced F1 score.

Shortcomings

For Logistic Regression, although it has the best overall performance, and is efficient with sparse features, it falls short when it comes to non-linear relationships or imbalanced classes.

When it comes to Random Forest, it excels at capturing non-linear patterns but has the potential to be overfitting towards dominant classes.

Naïve Bayes is the worst performing of all these models, with its strengths lying in its ease of use and speed. However, it falls short in this case due to its tendency to oversimplify the complex language structure.

Conclusion

Logistic Regression had the highest F1 score and nearly matched Random Forest on Kappa score, however Random Forest showed slightly better agreement with the true scores. Naïve Bayes underperformed both these models significantly. This highlights the limitations of classic models with TF-IDF features, which tend to ignore essay structure and argument coherence. Another limitation we came across was having a small sample size for scores 1 & 6. Further improvements could include using transformer-based embeddings like BERT for sentence embeddings, improved feature extraction around grammar and coherence, or ordinal regression to capture score scales more accurately.

References

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement, 20*(1), 37–46. https://doi.org/10.1177/001316446002000104

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., … & Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Jurafsky, D., & Martin, J. H. (2021). *Speech and language processing* (3rd ed. draft). Retrieved from https://web.stanford.edu/~jurafsky/slp3/

Loper, E., & Bird, S. (2002). NLTK: The Natural Language Toolkit. In *Proceedings of the ACL-02 Workshop on Effective Tools and Methodologies for Teaching Natural Language Processing and Computational Linguistics* (pp. 63–70). https://doi.org/10.3115/1118108.1118117

McCallum, A., & Nigam, K. (1998). A comparison of event models for naive Bayes text classification. In *AAAI-98 Workshop on Learning for Text Categorization*. Retrieved from http://www.cs.cmu.edu/~knigam/papers/multinomial-aaaiws98.pdf

McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 51–56). https://doi.org/10.25080/Majora-92bf1922-00a

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press. https://nlp.stanford.edu/IR-book/Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://www.jmlr.org/papers/v12/pedregosa11a.html

Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and statistical modeling with Python. In *Proceedings of the 9th Python in Science Conference* (pp. 57–61). https://conference.scipy.org/proceedings/scipy2010/pdfs/seabold.pdf

Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management, 45*(4), 427–437. https://doi.org/10.1016/j.ipm.2009.03.002

Waskom, M. L. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software, 6*(60), 3021. https://doi.org/10.21105/joss.03021

Zhang, Y., Jin, R., & Zhou, Z.-H. (2010). Understanding bag-of-words model: A statistical framework. *International Journal of Machine Learning and Cybernetics, 1*(1), 43–52. https://doi.org/10.1007/s13042-010-0001-0

The Learning Agency Lab. (2024). *Learning Agency Lab - Automated Essay Scoring 2.0* [Dataset]. Kaggle. https://www.kaggle.com/competitions/learning-agency-lab-automated-essay-scoring-2